

Write up sherlock PhantomRing

mô tả

Your organization's SOC team intercepted a suspicious binary during a routine threat hunting operation on a Linux server. The file was found in /var/tmp with an unusual name and was attempting to establish outbound connections. Initial analysis suggests this could be a post-exploitation agent. Your task is to perform static analysis on the binary to identify its capabilities, extract indicators of compromise, and understand the threat actor's infrastructure.

tool: kali , IDA pro

Sau khi giải nén file zip thu được file tên: agent

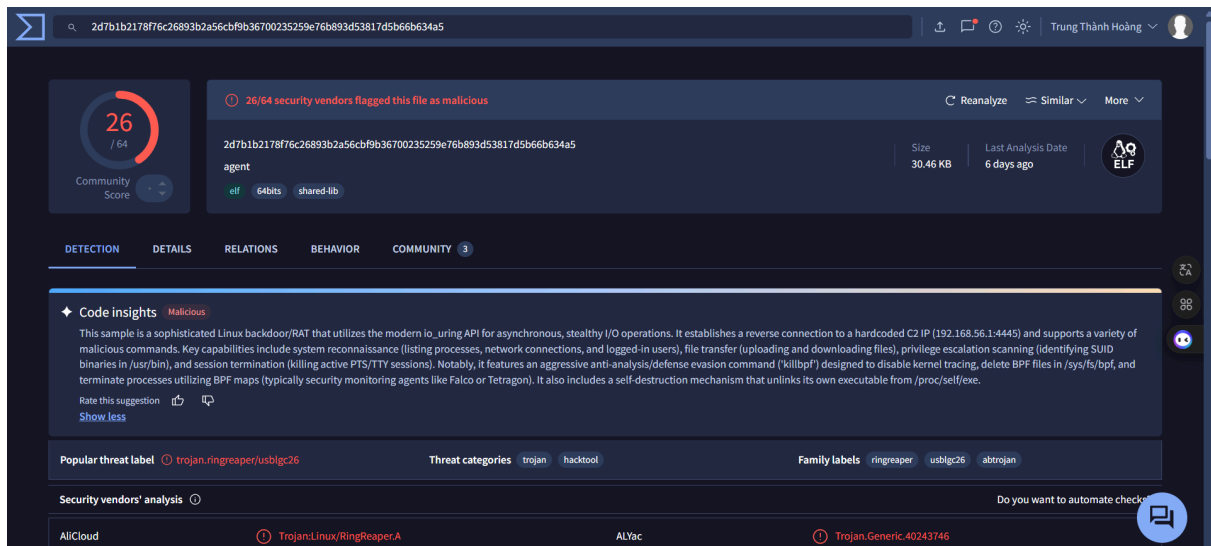
Name	Date modified	Type	Size
 agent	12/24/2025 3:38 AM	File	31 KB

mô tả đã gợi ý file đc tìm thấy ở /var/tmp rất có thể đây là 1 file malware nên tiến hành xem file đó là gì. điểm lưu ý : **not stripped** chứng tỏ hacker chưa xóa thông tin debug

```
(thanhbe@hacker)=[~/sherlock]
$ file agent
agent: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
724d7bbc01627982dc2f0495, for GNU/Linux 3.2.0, not stripped
```

task 1: What is the SHA256 hash of the malicious binary?

để biết được mã hash của file là gì ném file lên virus total



ANS:

2d7b1b2178f76c26893b2a56cbf9b36700235259e76b893d53817d5b66b634a5

ngoài ra thông tin ở đây còn cho biết đây là một RAT/Backdoor sử dụng API `io_uring` để ẩn mình, có khả năng trinh sát hệ thống, leo thang đặc quyền và kết nối ngược về một địa chỉ C2 được hardcoded là `192.168.56.1:4445`

Task2: What is the IP address hardcoded in the binary for C2 communication?

như phân tích ở trên ta biết được ip kết nối về c2

ANS: 192.168.56.1

Task3: What port does the agent connect to on the C2 server?

cũng như phân tích ở trên

ANS: 4445

Task4: How many seconds does the agent wait before attempting to reconnect after a failed connection?

Đến đây ta dùng IDA/Hydra để phân tích file mã độc này

theo như câu hỏi liên quan đến thời gian ta nghĩ ngay tới các hàm `sleep()`, `usleep()`, `nanosleep()`, `select()` hoặc là một nhánh xử lý lỗi của một lệnh gọi network `connect()`

Vào hàm main → alt+T sleep và ta đã có được time : 0x78u đổi HEX sang DEC=120s

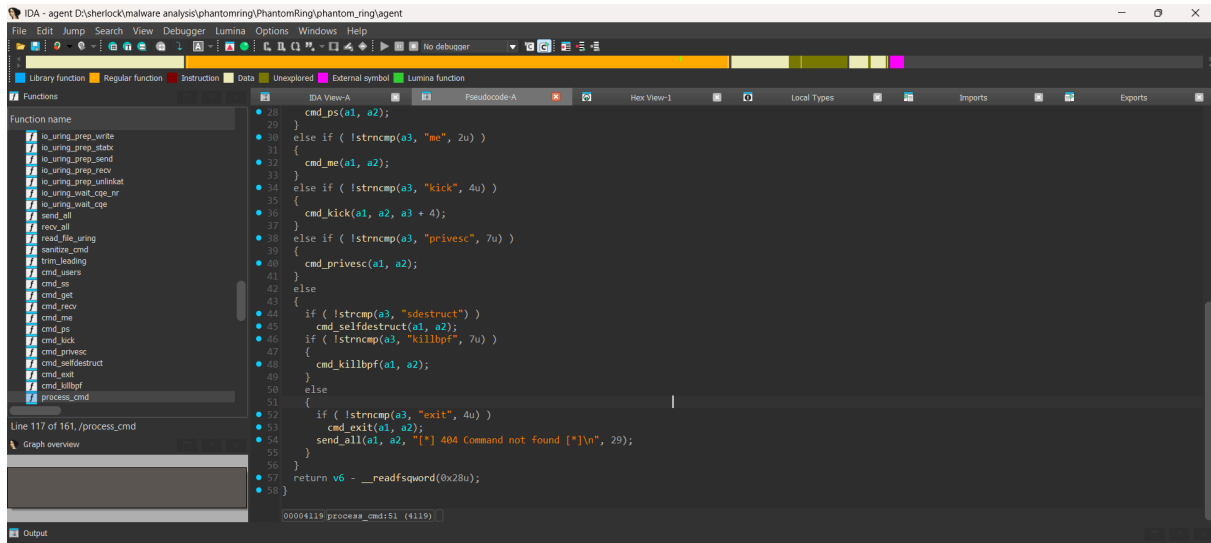
ANS:120

Task5:How many different commands does the agent support? (excluding invalid commands)

Để tìm được câu trả lời trước tiên hãy phân tích đoạn mã của hàm main

ta có thể thấy Buffer nhận dữ liệu là `v13` tham số của `io_uring_prep_recv` được truyền thẳng vào hàm `process_cmd`

tiếp theo vào hàm `process_cmd` để đọc. Tổng cộng có 11 hàm cmd



ANS: 11

Task6: What Linux kernel interface does this malware abuse to evade EDR syscall monitoring?

Nhìn lại hàm `main()` đa số luồng network I/O đều đi qua các hàm `io_uring_*` thay vì gọi socket API chuẩn trực tiếp

ANS: `io_uring`

Task7: What file does the agent read to enumerate logged-in users?

đề hỏi liên quan tới users nhìn lại vào hàm `process_cmd`

Hàm `process_cmd` đóng vai trò điều phối lệnh, nhận chuỗi lệnh từ qua

`io_uring_prep_recv`, so sánh bằng `strcmp` / `strncmp` với tập lệnh đã định nghĩa sẵn. Khi input khớp với chuỗi `"users"`, `process_cmd` gọi tới hàm `cmd_users` để thực thi logic thu thập users

```
IDA View-A Pseudocode-A Hex View-1
1 // cmd_users
2 unsigned __int64 __fastcall process_cmd(__int64 a1, unsigned int a2, const char *a3)
3 {
4     const char *v5; // [rsp+20h] [rbp-10h] BYREF
5     unsigned __int64 v6; // [rsp+28h] [rbp-8h]
6
7     v6 = __readfsqword(0x28u);
8     sanitize_cmd(a3);
9     if ( !strcmp(a3, "get ", 4u) )
10     {
11         cmd_get(a1, a2, a3 + 4);
12     }
13     else if ( !strcmp(a3, "recv ", 5u) )
14     {
15         v5 = a3 + 5;
16         trim_leading(&v5);
17         cmd_recv(a1, a2, v5);
18     }
19     else if ( !strcmp(a3, "users", 5u) )
20     {
21         cmd_users(a1, a2);
22     }
23     else if ( !strcmp(a3, "ss", 2u) || !strcmp(a3, "netstat") )
24     {
25         cmd_ss(a1, a2);
26     }
27     else if ( !strcmp(a3, "ps", 2u) )
28     {
29         cmd_ps(a1, a2);
30     }
31     else if ( !strcmp(a3, "me", 2u) )
32     {
33         cmd_me(a1, a2);
34     }
35 }
```

Chuyển qua hàm `cmd_users` để xem

```
Unexplored External symbol Lumina function
IDA View-A Pseudocode-A Hex View-1 Local Types
1 // bad sp value at call has been detected, the output may be wrong!
2 unsigned __int64 __fastcall cmd_users(__int64 a1, unsigned int a2)
3 {
4     size_t v2; // rax
5     int i; // [rsp+18h] [rbp-4034h]
6     int file_uring; // [rsp+1Ch] [rbp-4030h]
7     unsigned __int64 v6; // [rsp+24h] [rbp-4028h]
8     _BYTE v7[16]; // [rsp+3Ch] [rbp-4010h] BYREF
9     char v8; // [rsp+4Ch] [rbp-4000h] BYREF
10     char v9[4112]; // [rsp+203Ch] [rbp-2010h] BYREF
11     __int64 v10; // [rsp+304Ch] [rbp-1000h] BYREF
12     unsigned __int64 v11; // [rsp+4044h] [rbp-8h]
13
14     while ( &v10 != &v8 )
15     {
16         v11 = __readfsqword(0x28u);
17         file_uring = read_file_uring(a1, "/var/run/utmp", v7, 0x2000);
18         if ( file_uring > 0 )
19         {
20             // ...
21         }
22     }
23 }
```

ANS: /var/run/utmp

Task8:What directory does the agent scan when searching for SUID binaries for privilege escalation?

đề hỏi tới lệnh `privesc` ta sẽ vào hàm `cmd_privesc` để xem

```
unsigned __int64 v15; // [rsp+4344h] [rbp-8h]

while ( &v14 != &v13 )
{
    v15 = __readfsqword(0x28u);
    dirp = opendir("/usr/bin");
    if ( dirp )
    {
        v5 = snprintf(v12, 0x4000u, "Potential SUID binaries:\n");
        while ( 1 )
        {
            v7 = readdir(dirp);
            if ( !v7 )
                break;
            snprintf(s, 0x200u, "/usr/bin/%s", v7->d_name);
            sqe = io_uring_get_sqe(a1);
            io_uring_prep_statx(sqe, 4294967196LL, s, 0, 4095, v9);
        }
    }
}

000032D8 cmd_privesc:1 (32D8)
```

ANS: /usr/bin

Task9:What string does the agent search for in /proc/[pid]/maps to identify security tools using eBPF?

liên quan tới lệnh `killbpf` → vào hàm `cmd_killbpf` .Agent chắc chắn có 1 vòng lặp duyệt qua các PID trong `/proc` , đọc file `/proc/[pid]/maps` của từng process, rồi dùng `strstr()`

```

91 }
92 v21 = opendir("/proc");
93 if ( v21 )
94 {
95     while ( 1 )
96     {
97         v22 = readdir(v21);
98         if ( !v22 )
99             break;
100         if ( ((*__ctype_b_loc())[v22->d_name[0]] & 0x800) != 0 )
101         {
102             snprintf(v26, 0x100u, "/proc/%s/maps", v22->d_name);
103             if ( read_file_uring(a1, v26, s, 0x4000) > 0 )
104             {
105                 if ( strstr(s, "anon_inode:bpf-map") )
106                 {
107                     pid = atoi(v22->d_name);
108                     if ( pid > 1 )
109                     {
110                         if ( kill(pid, 9) )
111                         {
112                             v9 = __errno_location();
113                             v10 = strerror(*v9);
114                             v8 = snprintf(&v27[v17], 0x2000 - v17, "[-] Failed to kill PID %d: %s\n", pid, v10);
115                         }
116                     }
117                     else
118                     {
119                         ++v12;
120                         v8 = snprintf(&v27[v17], 0x2000 - v17, "[+] Killed PID using BPF: %d\n", pid);
121                     }
122                 }
123                 v17 += v8;
124             }
125         }
126     }
127 }

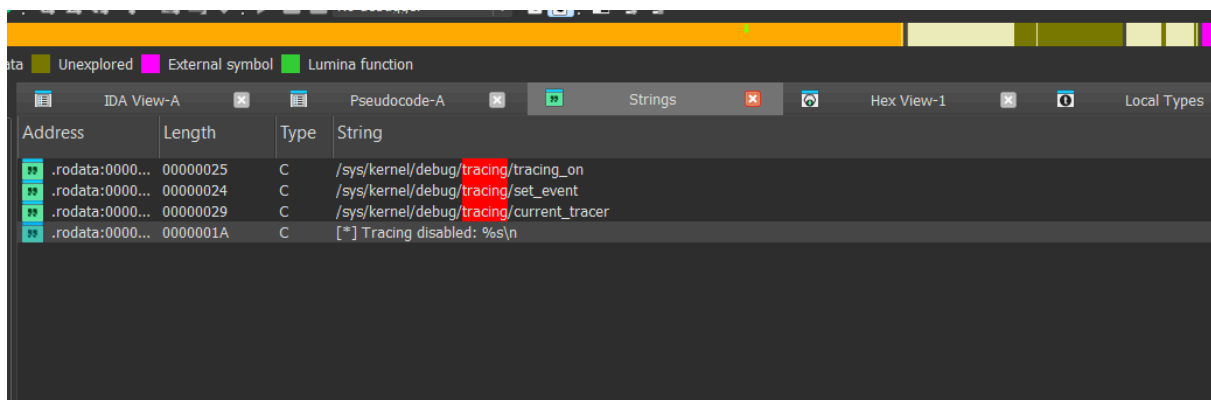
```

ANS: anon_inode:bpf-map

Task10: What is the full path of the first tracing file the agent attempts to disable?

đề bài nhắc tới tracing file search thì đây là một từ khóa liên quan đến thư mục trên linux `/sys/kernel/debug/tracing/` hoặc **eBPF tracing** (`bpftrace` , `bpf_trace`)

Vào strings tìm tracing ta thấy được các từ khóa chọn dòng cuối



offset `+59` được tham chiếu sớm nhất trong hàm

```

.rodata:00000000000052BB ; const char aUsrBinS[]
.rodata:00000000000052BB aUsrBinS db '/usr/bin/%s',0 ; DATA XREF: cmd_privesc+EC↑o
.rodata:00000000000052C7 ; const char aS[]
.rodata:00000000000052C7 aS db '%s',0Ah,0 ; DATA XREF: cmd_privesc+1BC↑o
.rodata:00000000000052CB aAgentWillSelfD db 'Agent will self-destruct',0Ah,0
.rodata:00000000000052CB ; DATA XREF: cmd_selfdestruct+2B↑o
.rodata:00000000000052E5 ; const char path[]
.rodata:00000000000052E5 path db '/proc/self/exe',0 ; DATA XREF: cmd_selfdestruct+76↑o
.rodata:00000000000052F4 ; const char aUnlinkFailedS[]
.rodata:00000000000052F4 aUnlinkFailedS db 'Unlink failed: %s',0Ah,0
.rodata:00000000000052F4 ; DATA XREF: cmd_selfdestruct+13A↑o
.rodata:0000000000005307 align 8
.rodata:0000000000005308 aAgentDisconnect db 'Agent disconnecting and exiting',0Ah,0
.rodata:0000000000005308 ; DATA XREF: cmd_exit+22↑o
.rodata:0000000000005329 align 10h
.rodata:0000000000005330 aSysKernelDebug db '/sys/kernel/debug/tracing/tracing_on',0
.rodata:0000000000005330 ; DATA XREF: cmd_killbpf+59↑o
.rodata:0000000000005355 align 8
.rodata:0000000000005358 aSysKernelDebug_0 db '/sys/kernel/debug/tracing/set_event',0
.rodata:0000000000005358 ; DATA XREF: cmd_killbpf+67↑o
.rodata:000000000000537C align 20h
.rodata:0000000000005380 aSysKernelDebug_1 db '/sys/kernel/debug/tracing/current_tracer',0
.rodata:0000000000005380 ; DATA XREF: cmd_killbpf+75↑o
.rodata:00000000000053A9 a0 db '0',0 ; DATA XREF: cmd_killbpf+171↑o
.rodata:00000000000053AB ; const char aTracingDisable[]
.rodata:00000000000053AB aTracingDisable db '["] Tracing disabled: %s',0Ah,0
.rodata:00000000000053AB ; DATA XREF: cmd_killbpf+257↑o

```

ANS: /sys/kernel/debug/tracing/tracing_on

Task11: What procfs path does the agent read to find its own executable location before self-destruction?

Câu hỏi liên quan tới hàm `cmd_selfdestruct` nên ta sẽ vào hàm để xem

```

1 void __fastcall __noreturn cmd_selfdestruct(__int64 a1, unsigned int a2)
2 {
3     size_t v2; // rax
4     char *v3; // rax
5     size_t v4; // rax
6     __QWORD v5[2]; // [rsp+10h] [rbp-2B0h] BYREF
7     ssize_t v6; // [rsp+20h] [rbp-2A0h]
8     __int64 sqe; // [rsp+28h] [rbp-298h]
9     char v8[128]; // [rsp+30h] [rbp-290h] BYREF
10    char buf[520]; // [rsp+B0h] [rbp-210h] BYREF
11    unsigned __int64 v10; // [rsp+2B8h] [rbp-8h]
12
13    v10 = __readfsqword(0x28u);
14    v5[1] = "Agent will self-destruct\n";
15    v2 = strlen("Agent will self-destruct\n");
16    send_all(a1, a2, "Agent will self-destruct\n", v2);
17    v6 = readlink("/proc/self/exe", buf, 0xFFFu) char[26]
18    if ( v6 > 0 )
19    {
20        buf[v6] = 0;

```

ANS: /proc/self/exe

Task12: What command string is compared by the agent to trigger deletion of its own binary?

liên quan đến so sánh ta lại quay về hàm `cmd_process` nơi so sánh chuỗi nhận từ operator bằng `strcmp / strncmp` nhìn vào toàn bộ src thì chỉ có đoạn này là if đơn lẻ

.Đây là dòng duy nhất trong toàn bộ hàm gọi tới `cmd_selfdestruct`

```
else
{
    if ( !strcmp(a3, "sdestruct") )
        cmd_selfdestruct(a1, a2);
    if ( !strcmp(a3, "killbpf" - 7u) )
```

ANS: sdestruct

tóm tắt quá trình malware chạy

Sau khi khởi chạy, agent khởi tạo hàng đợi `io_uring` và liên tục cố gắng kết nối tới máy chủ C2 tại `192.168.56.1:4445`; nếu kết nối thất bại, nó đóng socket, chờ 120 giây rồi thử lại, lặp vô thời hạn cho tới khi thành công. Khi kết nối được thiết lập, agent chuyển sang trạng thái chờ lệnh, liên tục nhận dữ liệu từ operator và chuyển vào bộ định tuyến lệnh (`process_cmd`) để so khớp và thực thi một trong 11 lệnh được hỗ trợ: thu thập thông tin hệ thống (liệt kê user đăng nhập, tiến trình đang chạy, kết nối mạng), thao tác file (nhận/gửi dữ liệu), leo quyền (quét SUID binary trong `/usr/bin`), can thiệp session người dùng khác, và đặc biệt là hai lệnh mang tính phòng thủ chủ động: vô hiệu hoá công cụ giám sát dựa trên eBPF/ptrace (tắt tracing, xoá BPF pinned file, kill tiến trình liên quan), và tự xoá binary khỏi đĩa thông qua `/proc/self/exe` để xoá dấu vết. Vòng lặp nhận-xử lý lệnh này tiếp diễn cho tới khi operator gửi lệnh thoát hoặc kết nối bị ngắt, lúc đó agent dọn dẹp tài nguyên `io_uring` và tự kết thúc tiến trình